

# 5

# SYNCHRONIZATION

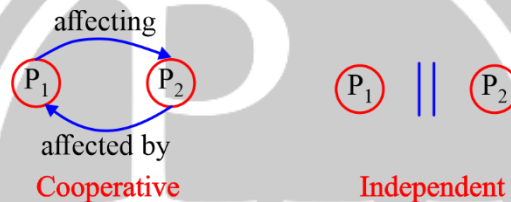
## 5.1 Synchronization

Processes are categorized into two types

Process



Execution of one process affects or affected by other process then those processes are said to be cooperative process. Otherwise, they are said to be Independent Process.



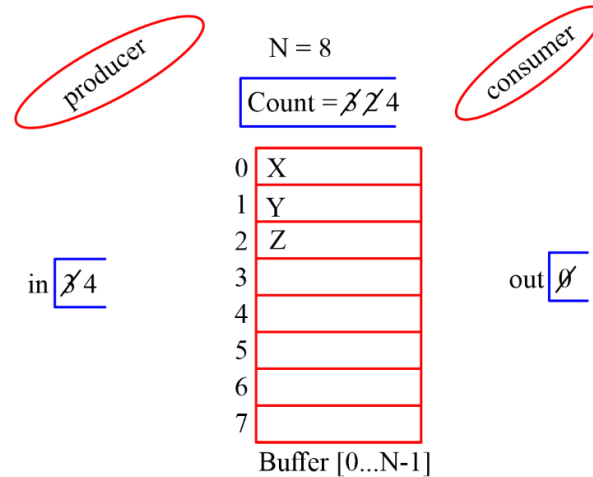
Processes must be cooperative to be synchronized.

## 5.2 Understanding Synchronization

- (1) The problems arise not having synchronization between the processes.
- (2) The conditions to be followed to achieve synchronization.
- (3) The solutions (wrong and right).

## 5.3 Problems arises for not having synchronization between the processes

### 5.3.1 Producer Consumer



int count = 0; // Initially buffer will be empty and only producer will be allowed to execute first.

void producer (void)

```
{
    int itemp;
    while(true)
    {
        produce_item(itemp);
        while (count == N); // Buffer Full
        Buffer [in] = itemp;
        in = (in + 1)modN; // ← to repeat in value from 0 - 7
        count = count + 1;
    }
}
```

Register of producer memory

- I. Load  $R_p.m[count]$
- II. INCR  $R_p$
- III. Store  $m[count], R_p$

void consumer(void)

```
{
    int itemc;
    while(true)
    {
        while (count == 0);
        itemc = Buffer [out];
        out = (out + 1) modN;
        count = count - 1;
        process_item(itemc);
    }
}
```

- I. Load  $R_c.m[count]$
- II. DECR  $R_c$
- III. Store  $m[count], R_c$

- IN is a variable used by the producer to identify the next empty slot in the buffer.
- OUT is a variable used by the consumer to identify from where it has to consume the item.

- Count is a variable used both producer and consumer to identify no of items present in the buffer at any point of time.
- Shared Resources:**
  - (1) Count Variable
  - (2) Buffer
- Two conditions to be followed:**
  - (1) If the buffer is full, producer is not allowed to produce the item into the buffer.
  - (2) If the buffer is empty, consumer is not allowed to consume the item from the buffer.

### 5.3.2 Universal Assumption

The running process can get pre-empted at any point of time after completion of current instruction.

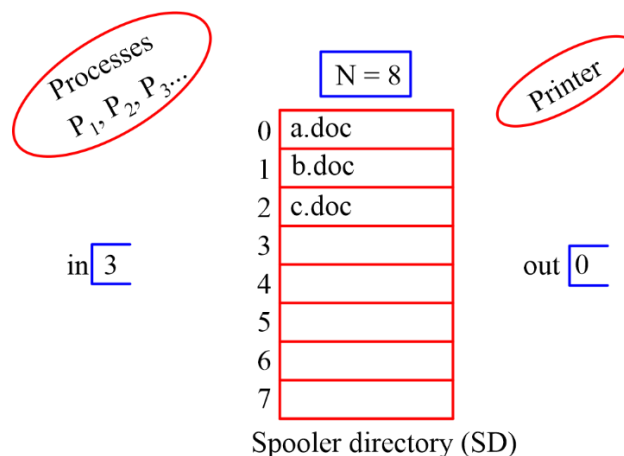
**Analysis:**

|             |                 |
|-------------|-----------------|
| Itemp = 'A' | count           |
| Itemc = 'X' | 3 4             |
| P → I       | Rp 3 in 4 out 1 |
| P → II      | Rp 4            |
| C → I       |                 |
| C → II      | Rc 2            |
| C → III     |                 |
| P → III     |                 |

Final count value = 4. Though actually three values are there.

- (1) Inconsistency:  
The producer and consumer are not properly synchronized while sharing the common variable “count”. Hence it is leading to the problem of inconsistency.

### 5.3.4 Printer Spooler Daemon (Daemon → Background process)



### 5.3.5 Definitions



**(1) Critical Section:**

The portion of program text where shared variables or shared resources will be placed.

**Example:**

Count = Count + 1  
OR  
Count = Count – 1

**(2) Non-Critical Section:**

The portion of program text where the independent code of the processes will be placed.

**Example:**

$T_n = (in + 1) \bmod N$

**(3) RACE condition:**

The final value of any variable depends on execution sequence of the processes. This type of condition is called as RACE condition.

- To avoid the RACE condition, only one process is allowed to enter into critical section.

### 5.3.6 Conditions to be followed to achieve Synchronization

**(1) Mutual Exclusion (M.E):**

- No two processes may be simultaneously present inside the critical section at any point of time.
- Only one process is allowed to enter into critical section at any point of time.

**(2) Progress:**

- No process running outside the critical section should block the other intersected process from entering into critical section when critical section is free.
- If there is only one process trying to enter into critical section then it should be definitely allowed to enter into critical section.
- If two or more processes are trying to enter into critical section then one process should be definitely allowed to enter critical section.

**(3) Bounded Waiting:**

- No process should have to wait forever to enter into critical section.
- There should be a bound on getting chance to enter into critical section.
- Some process is indefinitely waiting to enter into critical section because critical section is always busy by some other processes. This situation should not arise.
- If the bounded waiting is not satisfied then it is possible for starvation.

**(4) No assumptions related to hardware and the processor speed:**

- Number of processes.

**Solutions:**

**I. Software Type:**

- (a) Lock Variables
- (b) Strict alteration or Decker's Algorithm
- (c) Petersons Algorithm

**II. Hardware Type:**

- (a) TSL Instruction Set
- (b) Test and Set Lock

**III. OS Type:**

- (a) Counting semaphore
- (b) Binary Semaphore

#### IV. Programming Language Compiler Support Type:

- (a) Monitors

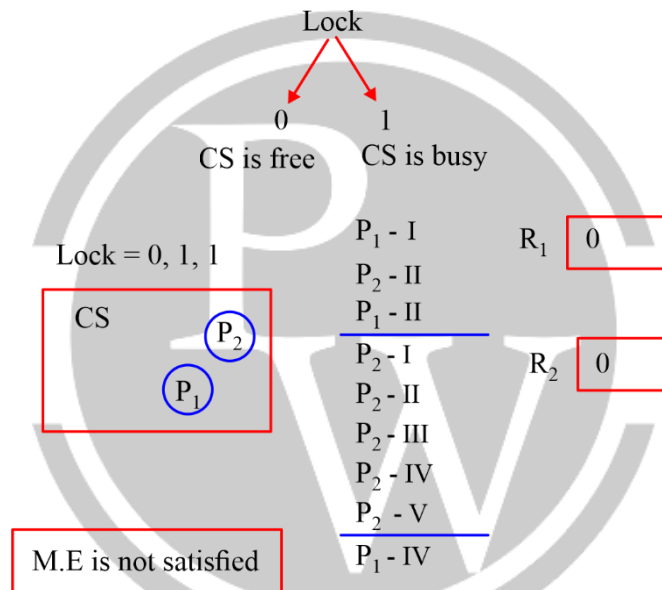
#### I. Software Types:

##### (a) Lock Variables:

Entry Section:

- Load  $R_i, m[\text{Lock}]$  ( $R_i \rightarrow$  Respective Process Register)
- Cmp  $R_i, \#0$
- JNZ to Step (I)
- Store  $m[\text{Lock}], \#1$
- C.S
- Store  $m[\text{Lock}], \#0$

#### Analysis:



We have proved that both the processes  $P_1$  and  $P_2$  are entering into the critical section at the same time, Hence mutual exclusion is not satisfied and the solution is bound to be incorrect.

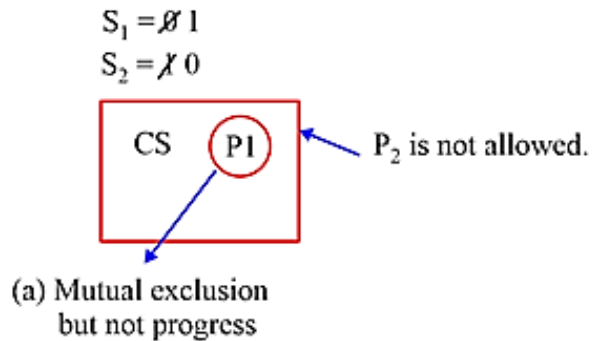
##### (b) Strict Alteration or Decker's Algorithm:

(Process takes 'Turn' to enter into C.S)

| Process ' $P_0$ ' code                                                                  | Process ' $P_1$ ' code                                                                |
|-----------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------|
| <pre>while (true) {     non_cs ();     while (turn! = 0);     c.s     turn = 1; }</pre> | <pre>while (true) {     non_cs();     while(turn! = 1);     c.s     turn = 0; }</pre> |

#### Important Points:

- The pre-emption is just a temporary stop and the process will come back and continue the remaining execution.
- If there is any possibility of solution becoming wrong by taking the pre-emption then consider the pre-emption.
- If any solution is having deadlock the progress is not satisfied.



### (c) Peterson's Algorithm:

(Two Process Solution)

```

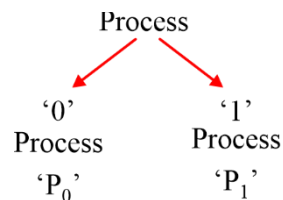
#define N 2
#define TRUE 1
#define FALSE 0
int turn;
int interested [N];
void enter_Region (int process)
{
    1. int other
    2. other = 1 - process;
    3. interested [process] = TRUE;
    4. turn = process;
    5. while (turn == process && interested [other] == TRUE);
}
C.S
void leave_Region(int process)
{
    interested [process] = FALSE;
}

```

initially

interested [0] = FALSE;

interested [1] = FALSE



TURN is a shared variable used by both the processes  $P_0$  and  $P_1$ , interested [N] is also shared by both the processes.

### 5.3.7 Hardware Type

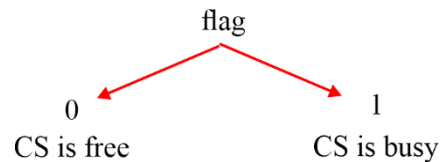
#### (a) TSL Instruction Set: (Test and Set Lock):

- **TSL Register Flag:**

Copies the current value of flag into register and stores the value of '1' into flag in a single atomic cycle without any pre-emption.

- **Entry Selection:**

1. TSL Ri, m[flag]
2. Cmp Ri, ≠ 0
3. JMP to step (1)
4. c.s
5. Store m[flag], ≠ 0



| Algorithm                                  | M.E. | Progress | Bounded Waiting |
|--------------------------------------------|------|----------|-----------------|
| 1. Lock Variable                           | ×    | ✓        | ×               |
| 2. Strict alteration or Decker's Algorithm | ✓    | ×        | ✓               |
| 3. Peterson's Algorithm                    | ✓    | ✓        | ✓               |
| 4. TSL Instruction                         | ✓    | ✓        | ×               |

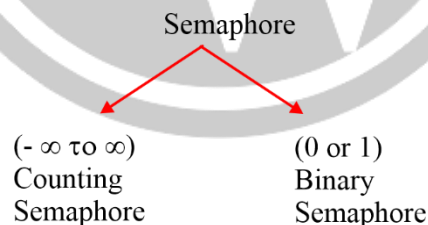
### 5.3.8 OS Type

#### Semaphore:

Semaphore is an integer variable which is used by the various processes in a mutual exclusive manner to achieve synchronization.

Improper usage of semaphore will also give the wrong results.

Semaphore is categorised into 2 types:



The two different operations will be performed on the semaphore variable.

- (1) Down, or wait (); or p ()
- (2) up (); or signal (); or v (); or release ();

#### (a) Counting Semaphore:

Down (Semaphore s)

```

{
    s.value = s.value - 1;
    if (s.value < 0)
    {
        Block the process and
        place its PCB in the
    }
}
  
```

```

        suspended list ();
    }
}
Up (Semaphore s)
{
    s.value = s.value + 1;
    if (s.value ≤ 0)
    {
        Select a process from
        suspended list and
        wakeup ();
    }
}

```

## (b) Binary Semaphore:

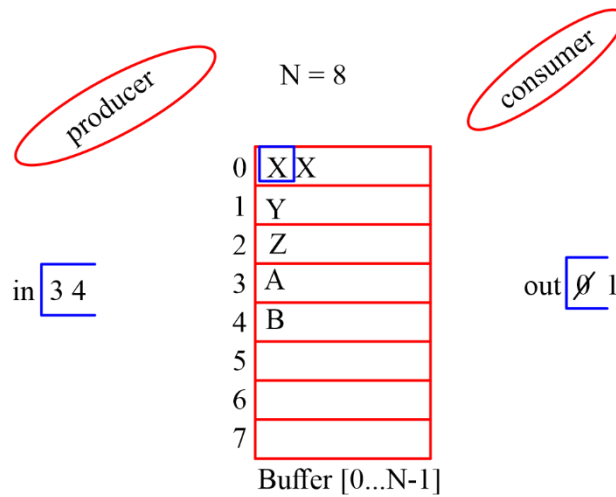
|                                                                                                                                                                                          |                                                                                                                                                                                          |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> Down (Semaphore S) {     if (S.value == 1)         S.value = 0;     else     {         block the process         and place its PCB         in the suspended list ()     } } </pre> | <pre> Up (Semaphore S) {     if (Suspended list is empty)         S.value = 1;     else     {         select a process         from suspended list         and wakeup ();     } } </pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

**Notes:** (Applicable for both counting and binary semaphore).

- ⇒ Every semaphore variable will have its own suspended list.
- ⇒ The down and up operations are atomic [OS will prevent the interrupts]
- ⇒ If more than one process is in the suspended list then every time when we perform one up operation one process will wakeup from the suspended list and this will be based on (FIFO → to ensure bounded waiting)
- ⇒ If two or more processes are in the suspended list and there is no other process to wakeup these processes then those processes are said to be involved in the deadlock.

## 5.4 Classical problems of IPC (Inter Process Communication)

- Producer consumer with semaphore:



```
semaphore mutex = 1;
```

```
semaphore empty = N;
```

```
semaphore full = 0;
```

```
void producer(void)
```

```
{
```

```
    int item p;
```

```
    while(true)
```

```
    {
```

```
        produce_item (item p);
```

```
        down(empty);
```

```
        down(mutex);
```

```
        buffer [in] = item p;
```

```
        in = (in + 1) mod N
```

```
        up(mutex);
```

```
        up(full);
```

```
    }
```

```
}
```

```
void consumer (void)
```

```
{
```

```
    int item c;
```

```
    while (true)
```

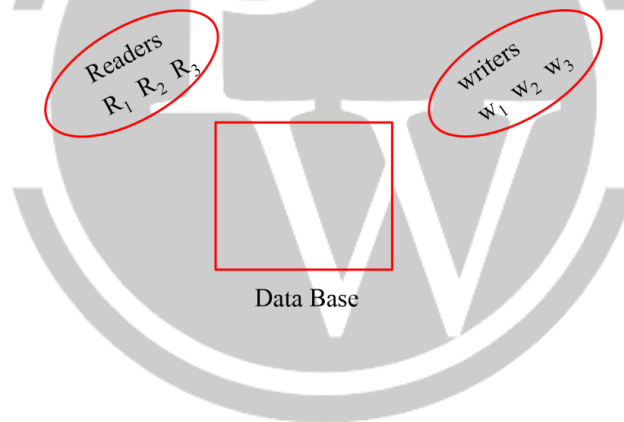
```

{
    down (full);
    down (mutex);
    item c = buffer [out];
    out = (out + 1)mod N;
    up (mutex);
    up (empty);
    process_item (item c);
}
}

```

- mutex is a binary semaphore used by the producer and consumer to access the buffer in a mutual exclusive manner.
- empty is counting semaphore variable represents number of empty slots in the buffer at any point of time.
- full is counting semaphore variable represents number of slots full in the buffer at any point of time.

## 5.5 READERS WRITERS PROBLEM



```

int rc = 0;
semaphore mutex = 1;
semaphore db = 1;
void reader (void)
{
    while (true)
    {
        down (mutex);
        rc = rc + 1;
        if (rc == 1) down (db);
        up (mutex);
    }
}

```

```

}

void writer (void)
{
    while (true){
        down (db);

        D.B

        up (db);
    }
}

```

## 5.5.1 conditions to be followed

- |            |            |
|------------|------------|
| 1) R - W ✗ | 2) R - R ✓ |
| 3) W - R ✗ | 4) W - W ✗ |

- rc is a integer variable represents readers count i.e number of readers present in the data base at any point of time.
- mutex is a binary semaphore used by the readers in mutual exclusive manner.
- db is a binary semaphore variable used by readers and writers in a mutual exclusive manner.

## 5.5.2 Dining Philosophers' problem

|  |                                                                                                                                                                                                                                                                                             |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <p><u>Dining Philosophers problem</u></p> <ul style="list-style-type: none"> <li>• N philosophers and N forks</li> <li>• Philosophers eat/think                     <ul style="list-style-type: none"> <li>▪ Eating needs 2 forks</li> <li>▪ Pick one fork at a time</li> </ul> </li> </ul> |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## 5.5.3 Solution 1 for Dining Philosophers problem

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> #define N5 /* number of philosophers */  Void philosophers (int i) /* i: philosophers' number from 0 to 4 */ {     while (TRUE) {         think (); /* philosopher is thinking */         take_fork (i); /* take left fork */         take_fork ((i+1)%N) /* take right fork; % is modulo operator */         eat (); /* yum-yum spaghetti */         put_fork (i); /* put left fork back on the table */         put_fork ((i+1)%N) /* put right fork back on the table */     } } </pre> | <ul style="list-style-type: none"> <li>• This solution to dining philosopher problem suffers from Deadlock.</li> <li>• Everyone picks up their left fork first, then waits for right fork... <math>\Rightarrow</math> this leads to Starvation!</li> </ul> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

- This solution to dining philosophers' problem suffers from Deadlock everyone picks up their left fork first, then waits for right fork  $\Rightarrow$  This leads to Starvation!

### 5.5.4 Solution 2 – state based

```
# define N                5                /* number of philosophers */
# define LEFT             (i+N-1)%N        /* number of I's left neighbours */
# define RIGHT            (i+1)%N          /* number of I's right neighbours */
# define THINKING         0                /* philosopher is thinking */
# define HUNGRY           1                /* philosopher is trying to get forks */
# define EATING           2                /* philosopher is eating */
typedef int semaphore      /*semaphores are a special kind of int */
int state [N];             /* array to keep track of everyone's state */
semaphore mutex = 1;       /* mutual exclusion for critical regions */
semaphore s[N];            /* one semaphore per philosopher */

void philosopher (int i)    /* i: philosopher number, from 0 to N -1 */
{                           /* repeat forever */
    Think();                /* philosopher is thinking */
    Take_forks(i);          /* acquire two forks or block */
    Eat();                  /* yum-yum spaghetti */
    Put_forks(i);           /* put both forks back on table */
}

void take_forks(int i)      /* i: philosopher number, from 0 to N -1 */
{
    down(&mutex);           /* enter critical region */
    state [i] = HUNGRY;     /* record fact that philosopher is hungry */
    test(i);               /* try to acquire two forks */
    up(&mutex);             /* exit critical region */
    down(&s[i]);             /* block if forks were not acquired */
}

void put_forks(i)          /* i: philosopher number, from 0 to N -1 */
{
    down(&mutex);           /* enter critical region */
    state [i] = THINKING;   /* philosopher has finished eating */
    test(LEFT);            /* see if left neighbour can now eat */
    test(RIGHT);           /* see if right neighbour can now eat */
    up(&mutex);             /* exit critical region */
}

void test(i)               /* i: philosopher number, from 0 to N -1 */
{
    if (state[i] == HUNGRY && state [LEFT] == EATING && state [RIGHT] == EATING) {
        state [i] = EATING;
        up(&s[i]);
    }
}
```

Made picking up left and right chopsticks an atomic operation  $\xi$  or,  $N - 1$  philosophers but  $N$  chopsticks  $\xi$  ...both changes prevent deadlock.

## 5.6 Monitors:

- Monitors is a programming language compiler support type of solution to achieve synchronization.
- Monitors is collection of variables and procedures combined together in a special kind of module or package.
- The process running outside the monitor cannot directly access the internal variables of the monitor but however they can call the procedures of the monitor.
- Monitors has an important property that only one process can be active inside the monitor at any point of time.

### 5.6.1 Syntax

Monitor example

```
{
    Variables;
    Condition variables;
    Procedure P1
    {
    }
    Procedure P2
    {
    }
}
```

### 5.6.2 Condition variables

Condition x,y;

The two different operations performed on the condition variables of the monitor.

- 1.Wait operation: wait ()
- 2.Signal operation: signal ()

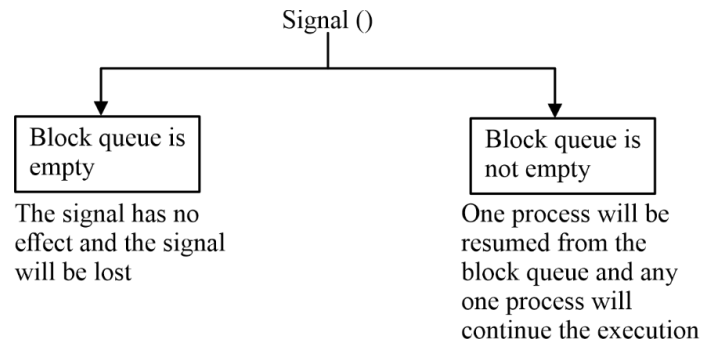
- **Wait ()**

Ex – x.wait (); or wait (x)

The process performing wait operation on any condition variable will be suspended and suspended process will be placed in the “block queue” of respective condition variable.

- **Signal ()**

Ex – x.signal (); or signal(x)



### 5.6.3 Concurrent Programming

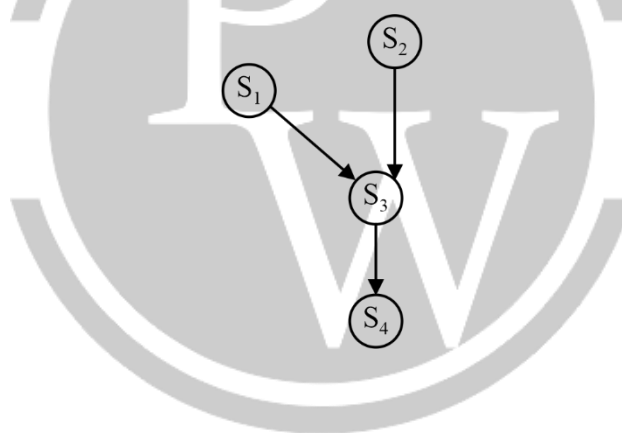
$S_1: a = b + c ;$   
 $S_2: d = e * f ;$   
 $S_3: g = a / d ;$   
 $S_4: h = g * i ;$

Read set = {b, c, e, f, a, d, g, i}

Write set = {a, d, g, h}

### 5.6.4 Precedence graph

$S_1, S_2$  can execute concurrently



- Any two statements  $S_i$  and  $S_j$  can be executed concurrently or parallel if they are following the conditions.

- (1)  $R(S_i) \cap W(S_j) = \varnothing$   
 (2)  $W(S_i) \cap R(S_j) = \varnothing$   
 (3)  $W(S_i) \cap W(S_j) = \varnothing$

- The real concurrent programming is possible only on the multiprocessor system.
  - Concurrent has 3 different meanings
    - They can execute concurrently or parallel
    - They don't have any dependency
- Anyone can start first [for single processor this will be applicable]

